

Campaign Oracle: TanStack Start Rewrite Proposal

This document outlines the architectural changes required to migrate "Campaign Oracle" to a full-stack **TanStack Start** application.

1. Project Structure

We will move from a flat `src` structure to the nested `app` structure required by TanStack Start.

Old Structure:

```
src/
├── services/
│   └── geminiService.ts <-- Client-side API call (Security risk)
├── App.tsx              <-- Monolithic UI
└── ...
```

New Structure:

```
app/
├── components/           <-- UI Components (Button, Input, etc.)
├── routes/
│   ├── __root.tsx       <-- Global Layout (Header, Footer, CSS)
│   └── index.tsx         <-- Main Campaign Page
├── server/
│   └── generate.ts        <-- Server Function (Safe API calls)
├── client.tsx            <-- Entry point
├── router.tsx            <-- Router config
└── ssr.tsx               <-- Server entry
```

2. Key Code Changes

A. The Server Function (`app/server/generate.ts`)

Instead of `geminiService.ts` running in the browser, we use a **Server Function**. This ensures `GEMINI_API_KEY` is never sent to the client.

```
import { createServerFn } from '@tanstack/start';
import { GoogleGenerativeAI } from '@google/generative-ai';
import { z } from 'zod'; // For input validation

// Define the input schema
const GenerateInputSchema = z.object({
  prompt: z.string(),
  // ... other schema fields ...
});
```

```

    tone: z.string().optional(),
  });

export const generateCampaignContent = createServerFn({ method: 'POST' })
  .validator((data) => GenerateInputSchema.parse(data))
  .handler(async ({ data }) => {
    // This code runs ONLY on the server
    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey) throw new Error("Missing API Key");

    const genAI = new GoogleGenerativeAI(apiKey);
    const model = genAI.getGenerativeModel({ model: "gemini-2.0-flash-exp" })

    const result = await model.generateContent(data.prompt);
    const response = await result.response;
    return response.text();
  });

```

B. The Main Route (app/routes/index.tsx)

We replace `App.tsx` with a route component. We use `useMutation` from TanStack Query (which is integrated) to handle the API state.

```

import * as React from 'react';
import { createFileRoute } from '@tanstack/react-router';
import { useMutation } from '@tanstack/react-query';
import { generateCampaignContent } from '../server/generate'; // Import the s

export const Route = createFileRoute('/')({
  component: CampaignGenerator,
});

function CampaignGenerator() {
  const [input, setInput] = React.useState('');

  // TanStack Query handles the loading/error states automatically
  const mutation = useMutation({
    mutationFn: async () => {
      // We call the server function just like a regular JS function
      return await generateCampaignContent({
        data: { prompt: input, tone: "professional" }
      });
    },
  });

  return (
    <div className="p-4 max-w-2xl mx-auto">
      <h1 className="text-2xl font-bold mb-4">Campaign Oracle</h1>

      <div className="flex gap-2 mb-4">
        <input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          type="text" />
        <button type="button" className="bg-blue-500 text-white px-4 py-2 rounded">Generate</button>
      </div>
    </div>
  );
}

```

```

        placenoider="Describe your campaign..."
        className="border p-2 rounded flex-1"
      />
      <button
        onClick={() => mutation.mutate()}
        disabled={mutation.isPending}
        className="bg-blue-600 text-white px-4 py-2 rounded disabled:opacit
      >
        {mutation.isPending ? 'Generating...' : 'Generate'}
      </button>
    </div>

    {mutation.isError ? (
      <div className="text-red-500">
        Error: {mutation.error.message}
      </div>
    ) : null}

    {mutation.data ? (
      <div className="bg-gray-100 p-4 rounded whitespace-pre-wrap">
        <h2 className="font-semibold mb-2">Result:</h2>
        {mutation.data}
      </div>
    ) : null}
  </div>
);
}

```

C. The Root Layout (app/routes/__root.tsx)

This replaces index.html and sets up the global providers.

```

import { Outlet, createRootRoute } from '@tanstack/react-router';
import { TanStackRouterDevtools } from '@tanstack/router-devtools';
import { Meta, Scripts } from '@tanstack/start';
import * as React from 'react';

export const Route = createRootRoute({
  component: RootComponent,
});

function RootComponent() {
  return (
    <html lang="en">
      <head>
        <Meta />
        <meta charSet="UTF-8" />
        <meta name="viewport" content="width=device-width, initial-scale=1.0" />
        <title>Campaign Oracle</title>
        {/* Tailwind CDN for quick setup, or import CSS file */}
        <script src="https://cdn.tailwindcss.com">(https://cdn.tailwindcss.co
      </head>
      <body>
        <Outlet /> {/* Renders the child route (index.tsx) */}
      </body>
    </html>
  );
}

```

```
        <scripts />
      </body>
    </html>
  );
}
```

3. Implementation Steps

1. Initialize Project:

```
npm create @tanstack/start@latest campaign-oracle
```

2. Install Dependencies:

```
npm install @google/generative-ai zod
```

3. Port Code:

- Copy the UI logic from your old `App.tsx` into `app/routes/index.tsx`.
- Move the logic from `services/geminiService.ts` into `app/server/generate.ts`.

4. Environment Variables:

- Create a `.env` file in the root.
- Add `GEMINI_API_KEY=your_key_here`.

4. Benefits of Rewriting

Feature	Old (Client-Side)	New (TanStack Start)
API Key Security	❌ Exposed in network tab	✅ Hidden on server
SEO	❌ Poor (Client Rendered)	✅ Excellent (SSR)
Data Fetching	⚠️ Manual <code>useEffect</code> / <code>fetch</code>	✅ <code>useMutation</code> / <code>useQuery</code>
Routing	⚠️ React Router DOM (Manual)	✅ File-system based (Automatic)